

Der Algorithmus im Überblick

Die Grundidee: Jedes Restaurant bekommt Punkte von 0 bis 100. Je besser es zu den Wünschen des Users passt, desto mehr Punkte. Wie viel ein Merkmal zählt, bestimmt der User mit den Wichtigkeits-Slidern. Am Ende wird sortiert, und das Restaurant mit den meisten Punkten steht oben.

Die Teile des Codes

1. Daten (restaurants.js)

Jedes Restaurant ist ein Eintrag mit Name, Küche, Preis, Bezirk, Adresse, Rating und Öffnungszeit. Das ist die „Datenbank“, die der Algorithmus durchsucht.

2. Eingaben des Users (Oberfläche in index.html)

Der Algorithmus bekommt zwei Pakete:

- **Wunsch:** gewählte Küche, maximaler Preis, Bezirk.
- **Gewichtung:** drei Zahlen von 0 bis 10, wie wichtig Küche, Preis und Bezirk sind.

3. Die Funktion `empfehleRestaurants` (algorithmus.js)

Das ist der eigentliche Algorithmus. Er arbeitet in vier Schritten:

Schritt	Was passiert
A. Gewichte umrechnen	Aus den Slider-Werten werden die maximalen Punkte pro Merkmal.
B. Erfüllung prüfen	Für jedes Restaurant wird geprüft, wie gut es zu jedem Wunsch passt (0 bis 1).
C. Punkte berechnen	Erfüllung × maximale Punkte, alles addiert.
D. Sortieren	Die Restaurants werden nach Punkten geordnet.

4. Filter „geöffnet“ (istOffen)

Vor der Berechnung werden alle Restaurants aussortiert, die laut Uhrzeit des Geräts gerade geschlossen sind. Geschlossene Restaurants bekommen also gar keine Punkte.

Die Rechnungen im Detail

A. Gewichte umrechnen

Die Summe aller Gewichte ergibt 100 Punkte, aufgeteilt nach Wichtigkeit:

$$\text{maximale Punkte} = \text{Gewicht} \div \text{Summe aller Gewichte} \times 100$$

Beispiel mit Küche 6, Preis 5, Bezirk 4 (Summe 15):

- Küche: $6 \div 15 \times 100 = \mathbf{40 \text{ Punkte}}$
- Preis: $5 \div 15 \times 100 = \mathbf{33,3 \text{ Punkte}}$
- Bezirk: $4 \div 15 \times 100 = \mathbf{26,7 \text{ Punkte}}$

Sind alle Gewichte 0, gibt es keine Punkte, damit nicht durch 0 geteilt wird.

B. Erfüllung pro Merkmal (Wert zwischen 0 und 1)

- **Küche:** passt = 1, passt nicht = 0.

- **Bezirk:** passt = 1, passt nicht = 0.
- **Preis:** Restaurant im Budget (gleich oder günstiger): 1; eine Stufe zu teuer (z. B. \$\$\$ bei Limit \$\$): 0,5; zwei Stufen zu teuer: 0.

Wählt der User „Egal“ bei Küche oder Bezirk, gilt das Merkmal automatisch als erfüllt.

C. Punkte berechnen

$$\text{Punkte} = (\text{Küche-Erfüllung} \times \text{Küche-Punkte}) + (\text{Preis-Erfüllung} \times \text{Preis-Punkte}) + (\text{Bezirk-Erfüllung} \times \text{Bezirk-Punkte})$$

Das Ergebnis wird auf eine ganze Zahl gerundet und als „87/100“ angezeigt.

Rechenbeispiel: Wunsch Italienisch, maximal \$\$, Bezirk 20, Gewichte 6 / 5 / 4.

Restaurant	Küche	Preis	Bezirk	Punkte
Osteria Augarten (Italienisch, \$\$, 20)	40	33,3	26,7	100
Ristorante Donaublick (Italienisch, \$\$\$, 20)	40	16,7 (halb)	26,7	83
Trattoria Stephansplatz (Italienisch, \$, 1)	40	33,3	0	73
Jadegarten (Chinesisch, \$\$, 20)	0	33,3	26,7	60

D. Sortieren und Küchen-Vorrang

Sortiert wird nach Punkten, absteigend. Eine Besonderheit: Hat der User eine Küche gewählt, stehen **alle Restaurants dieser Küche immer zuerst**, und die Punkte entscheiden nur innerhalb der Gruppen. Das haben wir eingebaut, weil sonst bei niedriger Küchen-Gewichtung ein chinesisches Restaurant vor italienischen stehen konnte. Mit Küche 1, Preis 10 und Bezirk 10 hätte „Goldener Drache“ (Chinesisch) 95 Punkte und „Osteria Augarten“ (Italienisch) nur 76. Wegen des Vorrangs steht trotzdem das italienische vorne.

Tipps für die Präsentation

- Starte mit dem Ablauf (Eingabe, Umrechnen, Bewerten, Sortieren) und zeige danach das Rechenbeispiel live: Ändere einen Slider und zeige, wie sich die Reihenfolge verändert.
- Erwähne die zwei Entscheidungen, die wir selbst getroffen haben: die halben Preis-Punkte bei einer Stufe über dem Limit und den Küchen-Vorrang. Das zeigt, dass der Algorithmus bewusst gestaltet ist.